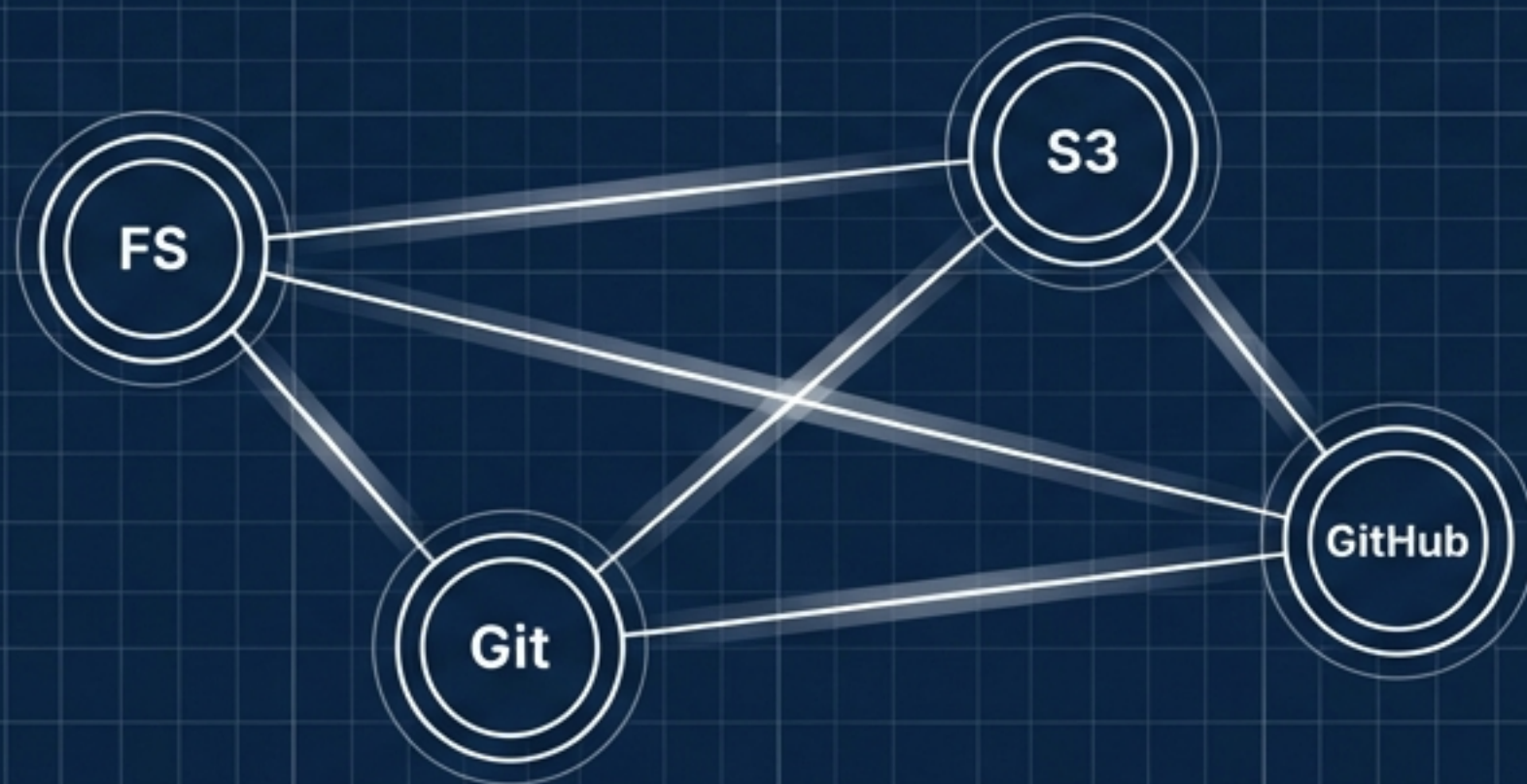


Issues-FS Ecosystem Architecture

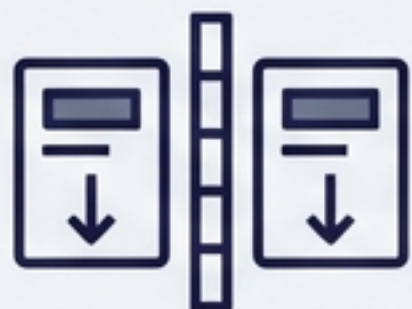
A file-system based issue tracking system designed for version control, offline access, and flexible deployment.



Version: v1.0
Date: 04 February 2026
Status: Active

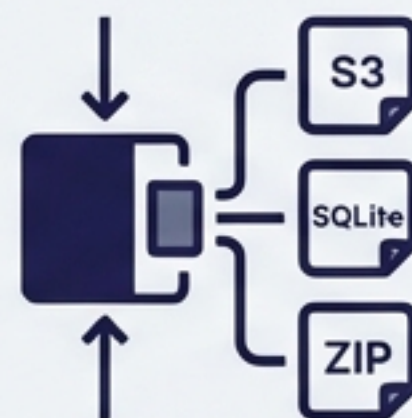
Core Design Principles

Issues-FS enables Git-based workflows and offline-first capabilities through a composable, modular architecture.



Separation of Concerns

Strict decoupling of the Core library from the Service layer.



Pluggable Storage

Implementation of the "memory-fs" abstraction allows data to live in memory, on disk, in S3, SQLite, or ZIP archives.



Type-Safety

All Python code enforces strict typing using "osbot-utils" Type_Safe patterns.



Service-Oriented

Each integration exposes its own independent FastAPI service.

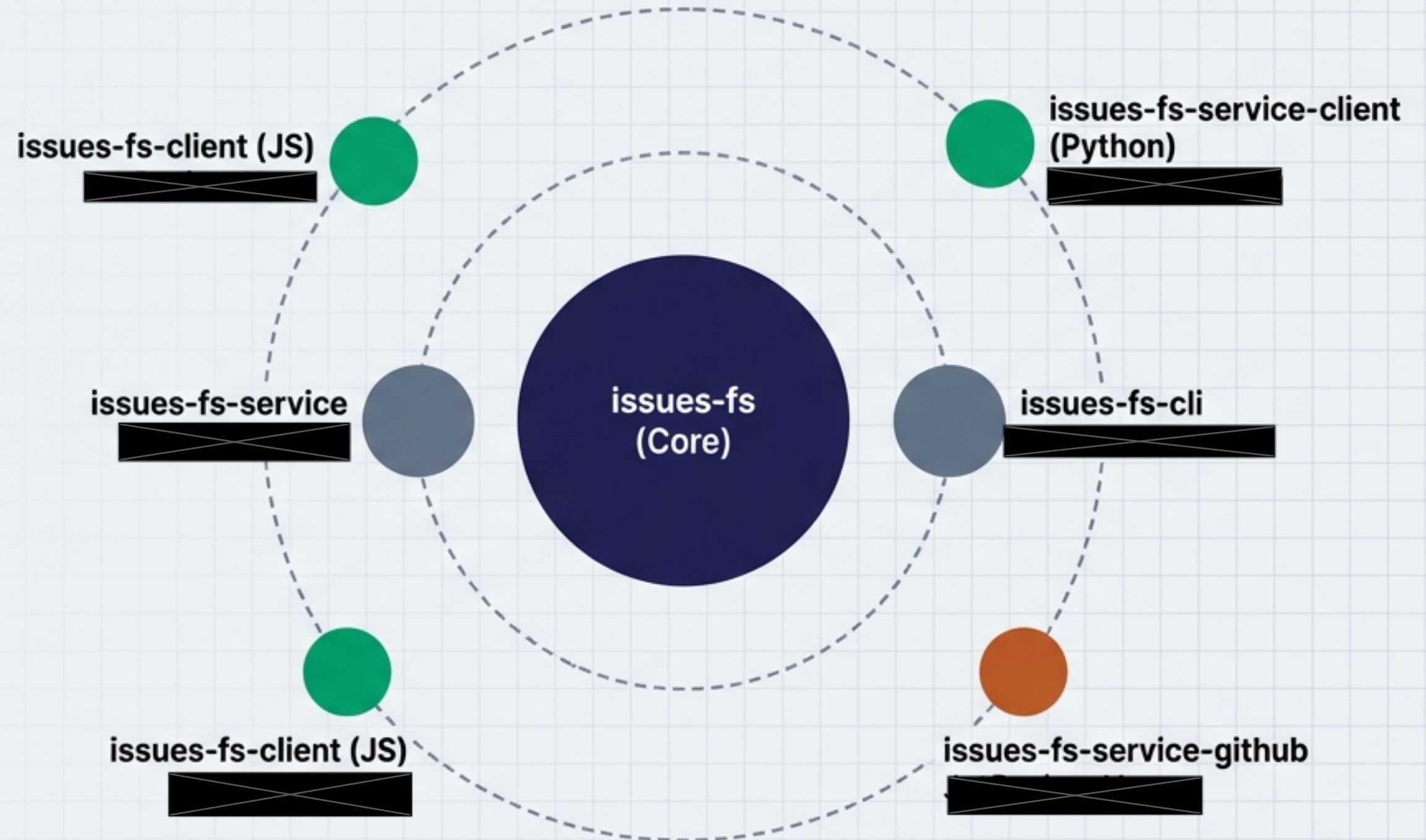
Ecosystem Topology

A galaxy view of the composable package structure

Key Insight: Focused Composability

The system is split into specialized packages following OSBot/MGraph-AI conventions.

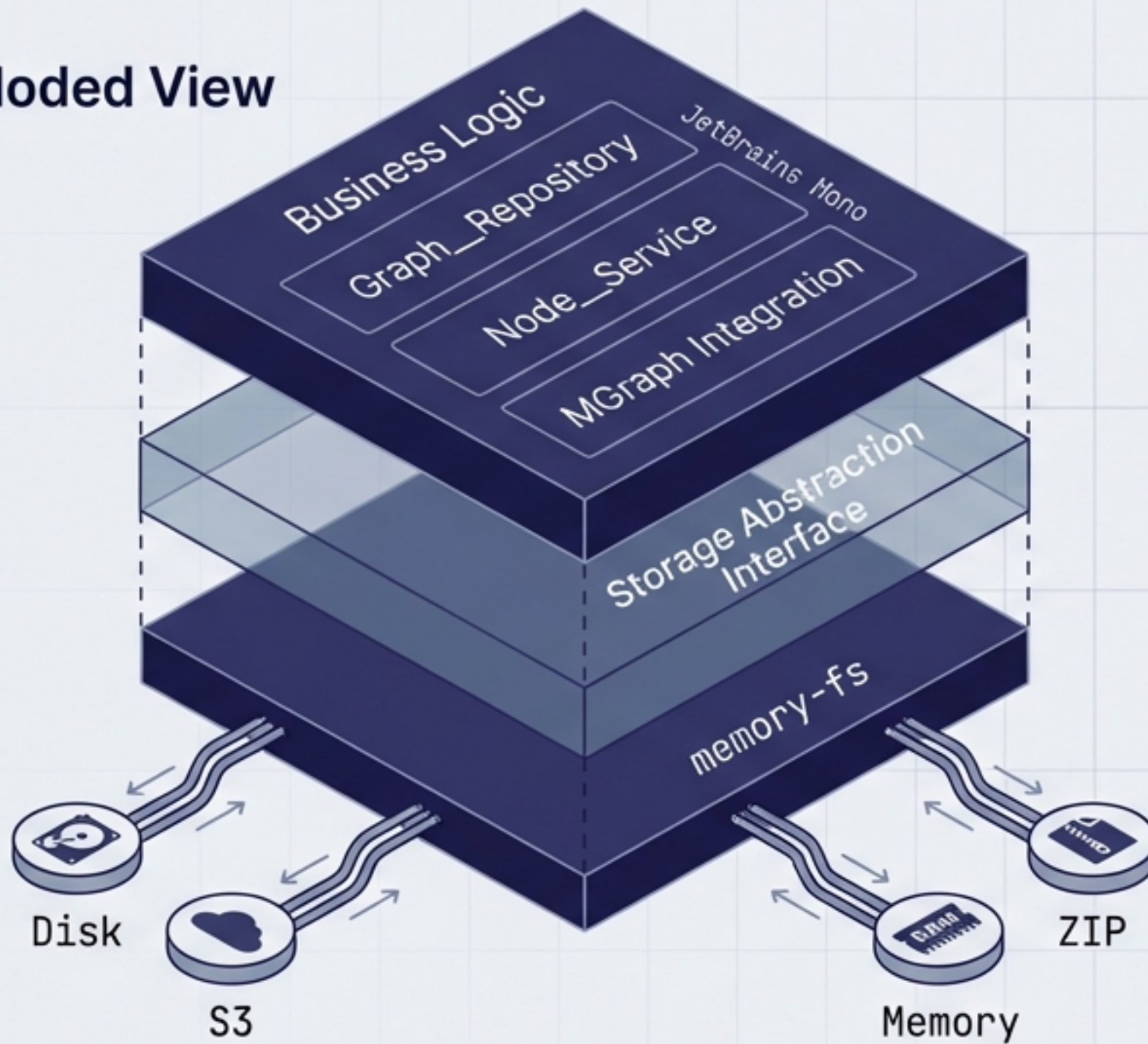
This separation allows the **Core** to function **independently** of the **API layer**, enabling direct CLI usage without server overhead.



The Bedrock: Core Library

Package: "issues-fs"

Exploded View



Technical Scope

The engine room where all business logic resides. Strictly separated from HTTP transport layers.

Dependencies

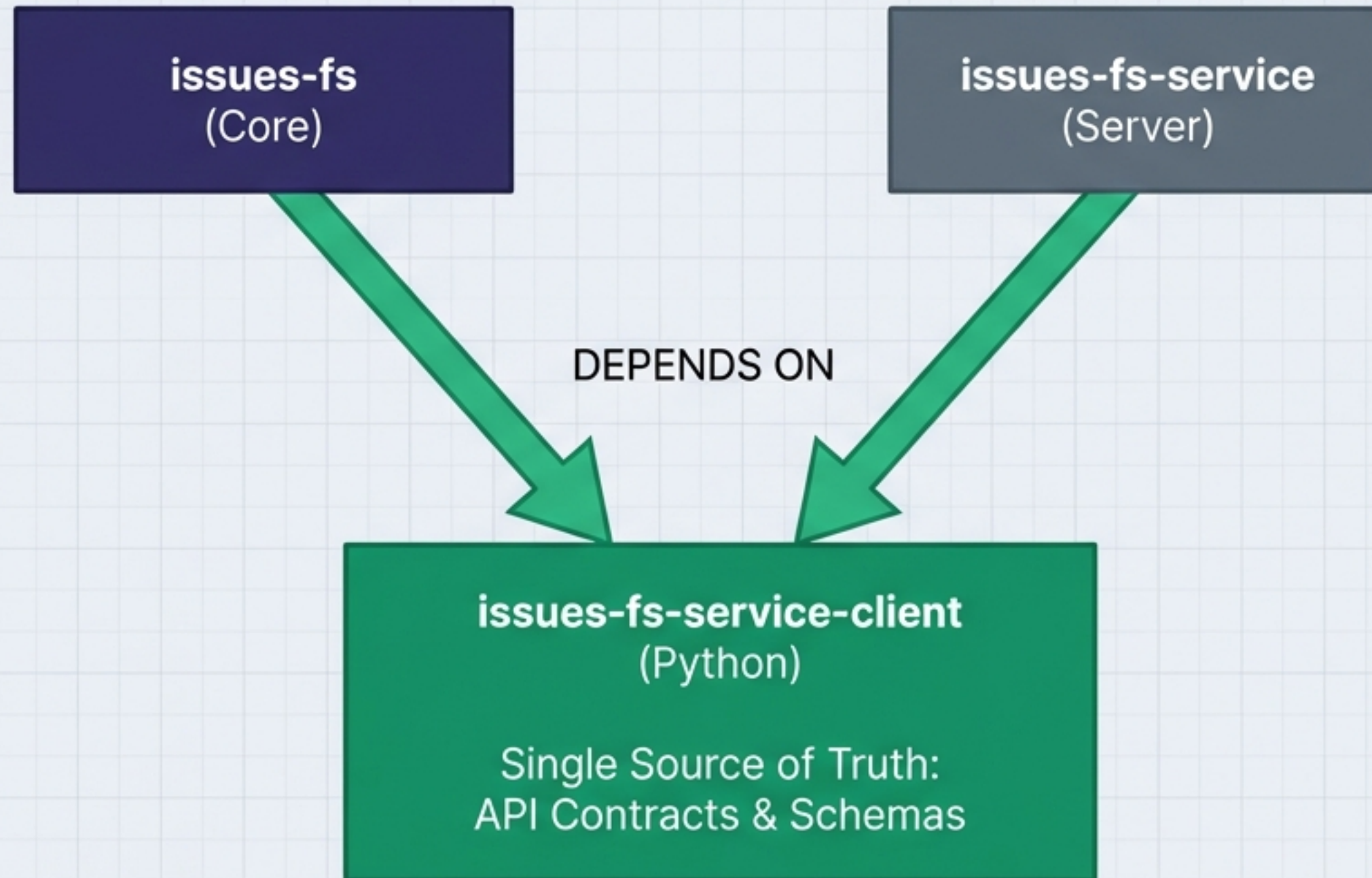
- 'mgraph-db' (Graph Engine)
- 'osbot-utils' (Utilities)

Consumers

- CLI Tools
- Service Server
- Standalone Python Apps

Architecture Spotlight: **The Schema Foundation**

The Key Insight: Decoupling Schemas from the Server.



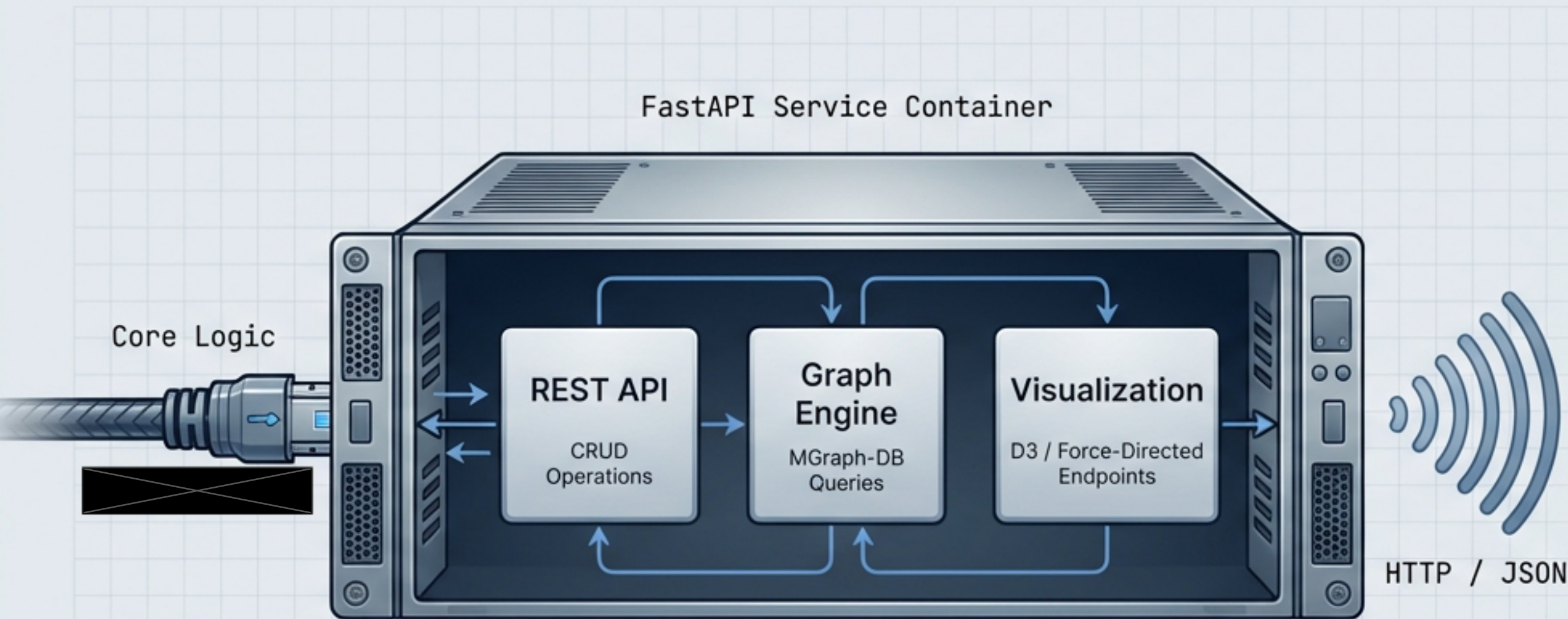
The Golden Rule

All Python schemas live in 'Issues-FS_Service_Client_Python'.

1. Prevents circular dependencies.
2. Ensures the Client is the definition of the API contract.
3. Guarantees type safety across the entire stack.

The Main Service

Package: "issues-fs-service"



****Context****

A FastAPI server acting as the bridge between the Core logic and external consumers.

****Tech Stack****

- FastAPI
- osbot-fast-api helpers

****Role****

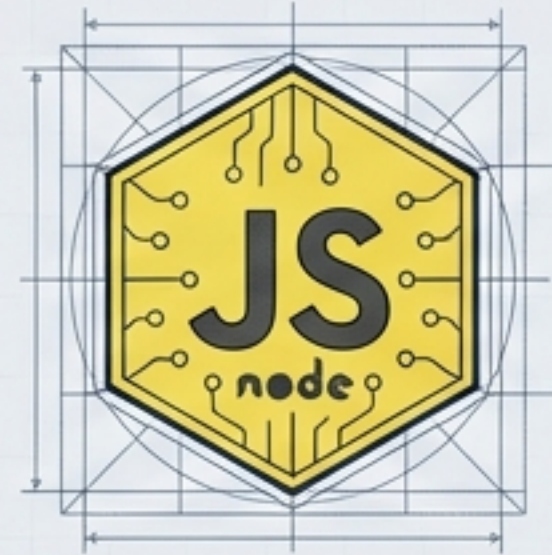
Connects the Core to the HTTP world using shared Schemas from the client package.

Access Points: Multi-Language Clients



Python Client
'issues-fs-service-client'

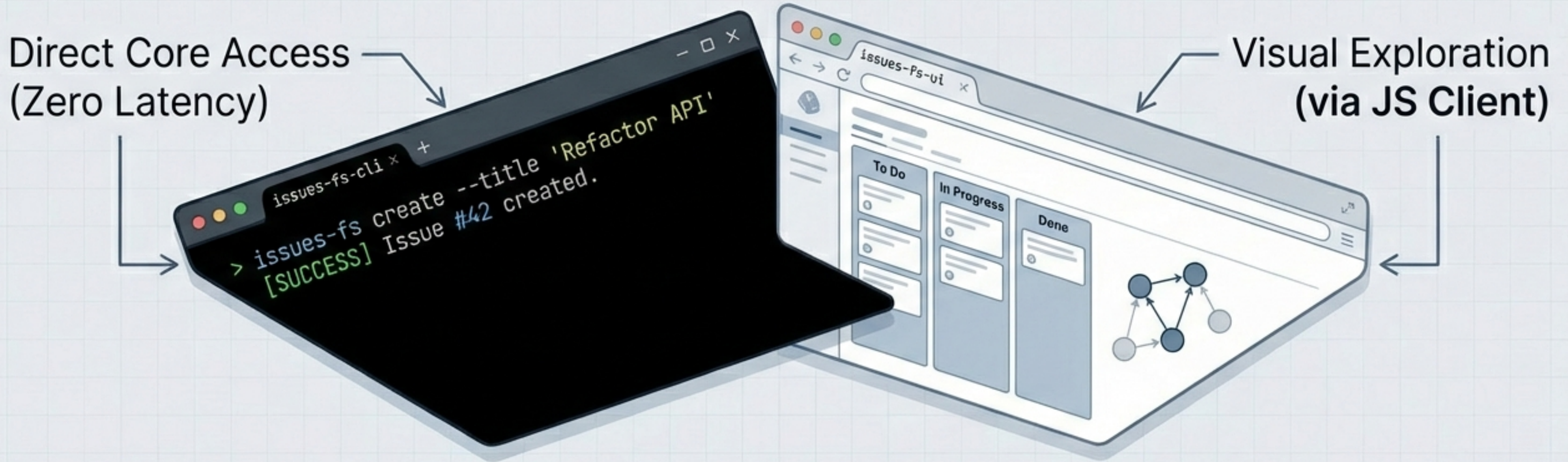
- **Role:** The Foundation.
- **Contents:** Contains all Schemas + 'requests'/'httpx' client.
- **Usage:** Used by GitHub integrations, server, and scripting.



JavaScript Client
'issues-fs-client'

- **Role:** The Frontend Bridge.
- **Registry:** Published to npm.
- **Contents:** TypeScript definitions and API client.
- **Consumers:** Electron apps, Node.js services, Web UI.

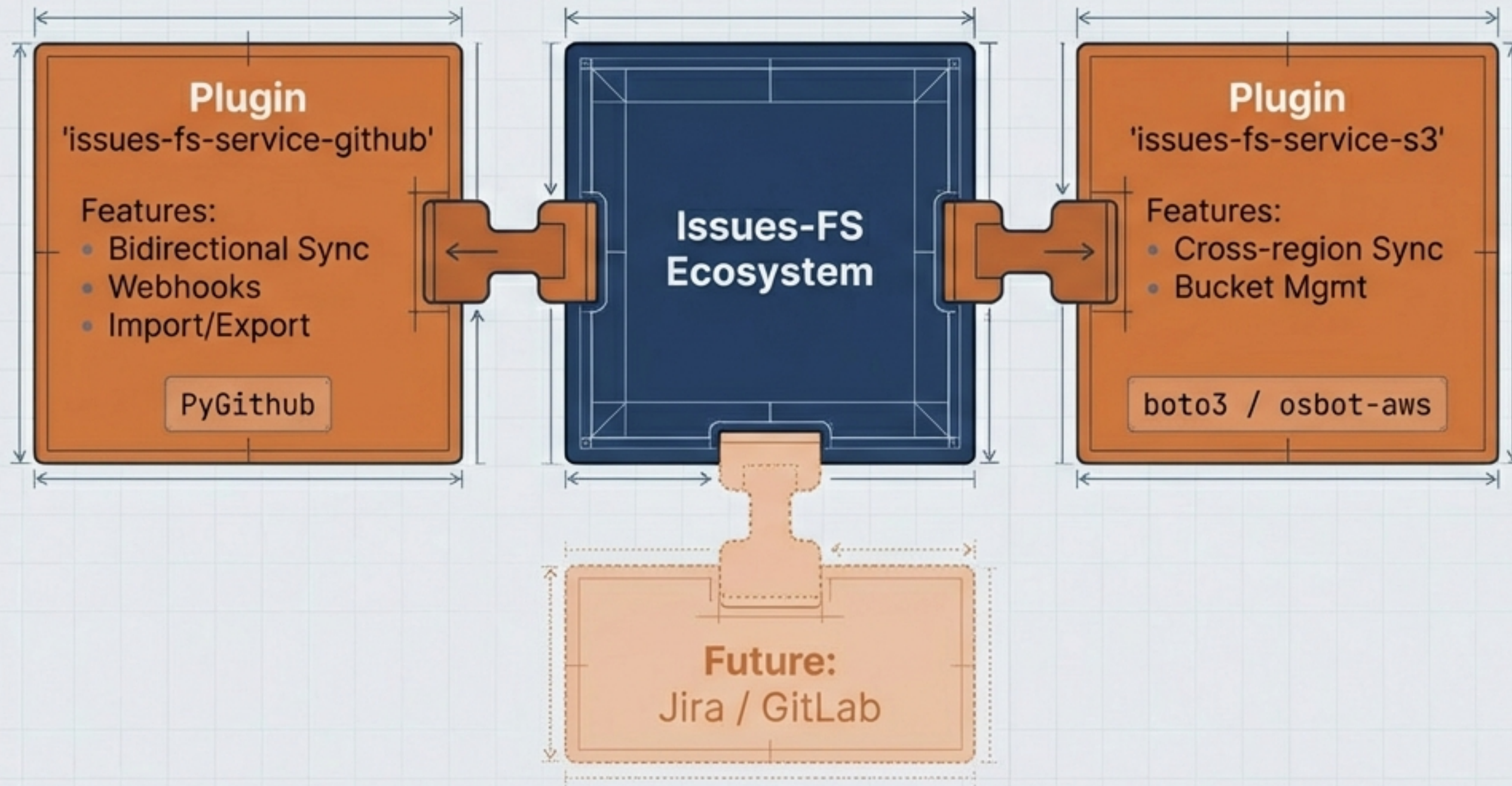
User Interfaces: Terminal & Web



Two modalities for interaction. The CLI provides rapid, scriptable management via direct library calls, while the Web UI offers hierarchical tree views, Kanban boards, and D3 graph **visualisations**.

Integration Services

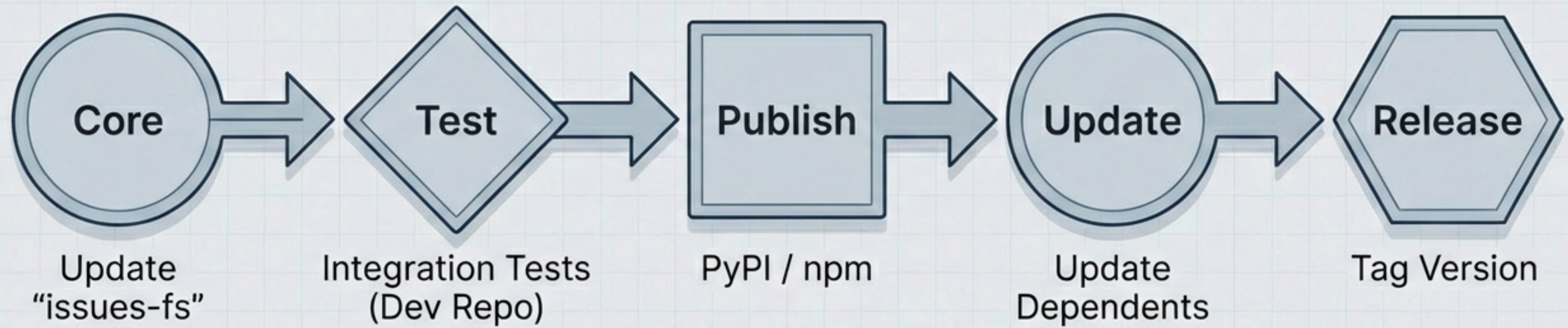
Connecting the ecosystem to the world.



Issues-FS does not live in a vacuum. Integrations run as independent services, attaching to the main ecosystem via the **Python Client** to sync, store, and adapt data.

Development Workspace

Repo: 'Issues-FS__Dev'



The Monorepo Strategy: A unified development environment uses submodules to manage the distributed package ecosystem, enabling integration tests and coordinated release scripts.

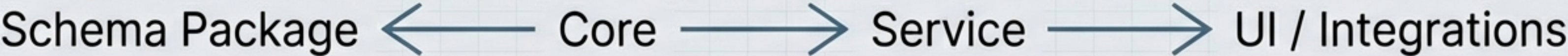
Package Registry & Dependencies

Table 1: PyPI Packages (Python)

Package Name	Repository	Role
"issues-fs"	"Issues-FS"	Core Library
"issues-fs-service"	"Issues-FS__Service"	API Server
"issues-fs-service-client"	"Issues-FS__Service__Client__Python"	Schemas & Client
"issues-fs-service-github"	"Issues-FS__Service__GitHub"	Integration

Table 2: npm Packages (JavaScript)

Package Name	Repository	Role
"issues-fs-client"	"Issues-FS__Service__Client__JS"	JS Client
"issues-fs-ui"	"Issues-FS__Service__UI"	Web Interface



Roadmap & Strategy

Future Integrations

- GitLab Sync Service
- Linear App Sync
- Notion Integration
- **Protocol Support**:**
“Issues-FS_MCP”
(Model Context
Protocol Server)

Versioning Strategy

// Versioning Rules

Core ('issues-fs'): Defines Semantic Versioning.
Breaking change = Major bump.

Service: Follows Core's Major version.

Clients: Strictly match Service API
version.

Integrations: Independent versioning.

Foundation References

**A robust, file-system based future
for issue tracking.**

Memory-FS	MGraph-DB	OSBot-Utils	OSBot-Fast-API
Storage Abstraction Layer	Graph Database Engine	Type_Safe Utilities	Service Helpers

Issues-FS Ecosystem Architecture v1.0
Repository Links Available in Documentation